

Codex Build Blueprint & Technical Specification

Dynamic Location-Based Solar Panel Tilt & Azimuth Web Application with Real-Time Weather API Integration

Target AI System: Codex / AI Code Generator

Stack: Streamlit / Python / Open-Meteo API

Document Type: System Architecture Spec

Primary Engine: ``scipy.optimize`` (SLSQP)

Data Sources: Open-Meteo & Nominatim Geocoding

Deployment: Single-File ``app.py``

Instruction for Codex / AI Coding Agent

Read this specification file thoroughly. Build a production-ready, fully interactive Python Streamlit web application (``app.py``) according to the UI/UX layout, API integration rules, mathematical optimization solver, and data visualization guidelines detailed below.

1. System Architecture & High-Level Flow

The web application enables users to enter any geographic location (City Name, Zip Code, or Latitude/Longitude coordinates), automatically pulls 7-day live weather forecasts and solar radiation data from free public APIs, and executes a Non-Linear Programming (NLP) optimization solver to compute the exact optimal daily and seasonal panel tilt (β) and azimuth (γ) angles.

Component	Technology / Service	Functionality
Frontend Framework	Streamlit (<code>`streamlit`</code>)	Renders responsive UI controls, interactive charts, metric cards, and download buttons.
Geocoding API	OpenStreetMap Nominatim API	Converts location strings (e.g., "Tempe, AZ") into Latitude and Longitude.
Weather API	Open-Meteo Weather & Solar API	Fetches 7-day hourly direct beam irradiance, diffuse sky radiation, and cloud cover.
Optimization Engine	SciPy Optimization (<code>`scipy.optimize.minimize`</code>)	Solves the continuous non-convex NLP using Sequential Quadratic Programming (SLSQP).
Visualizations	Plotly Express & Graph Objects	Displays 24-hour irradiance curves, 3D solar trajectory, and tilt comparison bar charts.

2. Embedded Mathematical Optimization Model

Codex must implement the mathematical formulation in the solver module:

Decision Variables

- β : Panel Tilt Angle in degrees, bounded by $10.0^{\circ} \leq \beta \leq 60.0^{\circ}$ (enforcing self-cleaning and wind limits).
- γ : Panel Azimuth Angle in degrees, bounded by $-45.0^{\circ} \leq \gamma \leq 45.0^{\circ}$ (0° = South, -90° = East, $+90^{\circ}$ = West).

Objective Function

Maximize Net Solar Energy Yield:
$$f(\beta, \gamma) = \sum_{h=1}^H [(I_{B,h} \cdot R_b(\beta, \gamma, h) + I_{D,h} \cdot ((1 + \cos \beta)/2) + I_{G,h} \cdot \rho \cdot ((1 - \cos \beta)/2)) \times (1 - C_{cloud,h}) \times (1 - L_{soiling}(\beta))]$$

- $I_{B,h}$, $I_{D,h}$, $I_{G,h}$: Direct Beam, Diffuse Sky, and Ground-Reflected irradiance fetched from Open-Meteo API for hour h .

- $C_{cloud,h}$: Cloud cover attenuation factor from live forecast.
- $L_{soiling}(\beta) = \alpha_0 \cdot \exp(-\lambda \cdot \beta)$: Empirical exponential dust penalty ($\alpha_0 = 0.12$, $\lambda = 0.18$).
- R_b : Geometric solar angle ratio based on latitude ϕ , solar declination δ , and hour angle ω .

3. Weather & Geocoding API Specifications

Codex should use `requests` to query the free Open-Meteo API without requiring an API key:

```
# API Endpoint URL Construction
GEOLOCATION_URL = "https://nominatim.openstreetmap.org/search?q={location_query}&format=json"
WEATHER_SOLAR_URL = "https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&hourly=direct_normal_irradiance,diffuse_radiation,cloudcover,temperature_2m&timezone=auto"
```

4. Required User Interface Components

1. Sidebar Inputs:

- Text Input for Location (Default: "Phoenix, AZ").
- Slider for Panel Surface Area ($\text{\$m}^2$, Default: 10.0 $\text{\$m}^2$).
- Slider for Panel Efficiency (% , Default: 20.0%).
- Radio selection for Adjustment Mode (Daily Optimal vs. Quarterly vs. Fixed Annual).

2. Top Key Performance Indicators (KPI Cards):

- Optimal Tilt Angle (η°) with deviation from Latitude.
- Optimal Azimuth Angle (γ°).
- Predicted Daily Energy Output ($\text{\$ kWh/day}$).
- Efficiency Gain (%) compared to standard flat installation.

3. Interactive Visualizations:

- **Line Chart:** Hourly Expected Solar Irradiance ($\text{\$/m}^2$) for Today (Fixed vs. Optimized).
- **Bar Chart:** 7-Day Predicted Energy Generation comparison.
- **Heatmap:** Surface plot showing Energy Output as a function of Tilt (η°) and Azimuth (γ°).

4. Export Feature:

Button to download a CSV summary of hourly recommendations and angle settings.

5. Complete Streamlit Code Scaffold for Codex Execution

```
import streamlit as st
import requests
import numpy as np
import pandas as pd
from scipy.optimize import minimize
import plotly.express as px
import plotly.graph_objects as go

st.set_page_config(page_title="AI Solar Panel Tilt Optimizer", layout="wide")

st.title("☀️ Dynamic Solar Panel Tilt & Orientation Optimizer")
st.markdown("Enter a location to fetch live weather & solar radiation forecasts to calculate optimal tilt.")
```

```

# Sidebar Controls
st.sidebar.header("User Inputs")
location_input = st.sidebar.text_input("Enter Location", "Phoenix, AZ")
panel_area = st.sidebar.slider("Panel Area (m²)", 1.0, 100.0, 15.0)
efficiency = st.sidebar.slider("Panel Efficiency (%)", 10.0, 25.0, 20.0) / 100.0

# 1. Geocoding Function
@st.cache_data
def get_lat_lon(location_str):
    url = f"https://nominatim.openstreetmap.org/search?q={location_str}&format=json"
    headers = {"User-Agent": "SolarOptimizerApp/1.0"}
    res = requests.get(url, headers=headers).json()
    if res:
        return float(res[0]['lat']), float(res[0]['lon']), res[0]['display_name']
    return 33.4484, -112.0740, "Phoenix, Arizona (Default)"

lat, lon, display_name = get_lat_lon(location_input)
st.sidebar.success(f"📍 Location: {display_name}")

# 2. Weather Fetching Function
@st.cache_data
def fetch_solar_weather(lat, lon):
    url = f"https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}&hourly=direct_normal_irradiance,diffuse_radiation,cloudcover&timezone=auto"
    res = requests.get(url).json()
    df = pd.DataFrame(res['hourly'])
    return df

weather_df = fetch_solar_weather(lat, lon)

# 3. Optimization Solver
def optimize_tilt_azimuth(lat, weather_df):
    def objective(vars):
        beta, gamma = vars[0], vars[1]
        soiling_loss = 0.12 * np.exp(-0.18 * beta)

        # Mean solar irradiance calculation across forecast
        avg_dni = weather_df['direct_normal_irradiance'].mean()
        avg_diffuse = weather_df['diffuse_radiation'].mean()

        # Trigonometric incidence model approximation
        cos_theta = np.cos(np.radians(lat - beta)) * np.cos(np.radians(gamma))
        total_irradiance = (avg_dni * max(0, cos_theta) + avg_diffuse * ((1 + np.cos(np.radians(beta))))/2))

        net_energy = total_irradiance * (1 - soiling_loss)
        return -net_energy # Maximize

    bounds = [(10.0, 60.0), (-45.0, 45.0)]
    res = minimize(objective, x0=[lat, 0.0], method='SLSQP', bounds=bounds)
    return res.x[0], res.x[1]

opt_beta, opt_gamma = optimize_tilt_azimuth(lat, weather_df)

# KPI Metrics Output
col1, col2, col3, col4 = st.columns(4)
col1.metric("Optimal Tilt Angle ( $\beta$ )", f"{opt_beta:.1f}°", f"{opt_beta - lat:+.1f}° vs Lat")
col2.metric("Optimal Azimuth ( $\gamma$ )", f"{opt_gamma:.1f}°", "0° = South")
col3.metric("Latitude", f"{lat:.2f}° N")
col4.metric("Forecast Days", "7 Days Live")

```

```
st.info("💡 PDF Blueprint generated successfully for Codex workspace import.")
```